

Windows-style root folder, `C:\`. If you're entering the interactive shell examples on macOS or Linux, enter `/` instead.

Additional volumes, such as a DVD drive or USB flash drive, will appear differently on different operating systems. On Windows, they appear as new, lettered root drives, such as `D:\` or `E:\`. On macOS, they appear as new folders within the `/Volumes` folder. On Linux, they appear as new folders within the `/mnt` (“mount”) folder. Note that folder names and filenames are not case sensitive on Windows and macOS, but they're case sensitive on Linux.

Paths in Python

On Windows, the backslash (`\`) separates folders and filenames, but on macOS and Linux, the forward slash (`/`) separates them. Instead of writing code both ways to make your Python scripts cross-platform compatible, you can use the `pathlib` module and `/` operator instead.

The typical way to import `pathlib` is with the statement `from pathlib import Path`. Because the `Path` class is the most frequently used class in `pathlib`, this form lets you type `Path` instead of `pathlib.Path`. You can pass a string of a folder or filename to `Path()` to create a `Path` object of that folder or filename. As long as the leftmost object in an expression is a `Path` object, you can use the `/` operator to join together `Path` objects or strings. Enter the following into the interactive shell:

```
>>> from pathlib import Path
>>> Path('spam') / 'bacon' / 'eggs'
WindowsPath('spam/bacon/eggs')
>>> Path('spam') / Path('bacon/eggs')
WindowsPath('spam/bacon/eggs')
>>> Path('spam') / Path('bacon', 'eggs')
WindowsPath('spam/bacon/eggs')
```

Note that because I ran this code on a Windows machine, `Path()` returns `WindowsPath` objects. On macOS and Linux, a `PosixPath` object is returned. (POSIX is a set of standards for Unix-like operating systems and is beyond the scope of this book.) For our purposes, there's no difference between these two types.